

7. Classification and regression trees

18.11.2025

ETH zürich

D MATH

Outline

- What are tree methods?
- Why use tree methods?
- Main part: How to fit trees to data (CART algorithm)
 - General structure of algorithm
 - Iteratively growing trees
 - Regularization, stopping criteria and pruning

Time estimate: ~90min

What are trees?

- For the purpose of machine learning, decision trees are functions where the input x is transformed into the output y via a finite sequence of simple decisions.
→ We focus on yes-or-no decisions. That is, we focus on **binary trees**
- In the context of classification and regression, tree-based models were popularized around the 1980s, see, e.g., the book "Classification And Regression Trees" by Breiman, Friedman, Olshen and Stone from 1984.
- On the one hand, CART simply means "Classification and Regression Trees", but on the other hand, CART is a particular algorithmic approach to learn certain kinds of binary trees from data
- The CART approach is the most popular tree-method and also the focus of these slides
- While decision/regression trees themselves are not the most powerful of methods, there are two major benefits of trees:
 - ① Binary trees are easy to interpret and explain, and thus attractive methods if the employed model needs to be trusted (e.g., medicine) or has to adhere to certain rules (e.g., insurance pricing)
 - ② Combined with model aggregation or ensemble learning techniques (like bagging and boosting), tree-based models can be very powerful

Motivating example: Binary classification

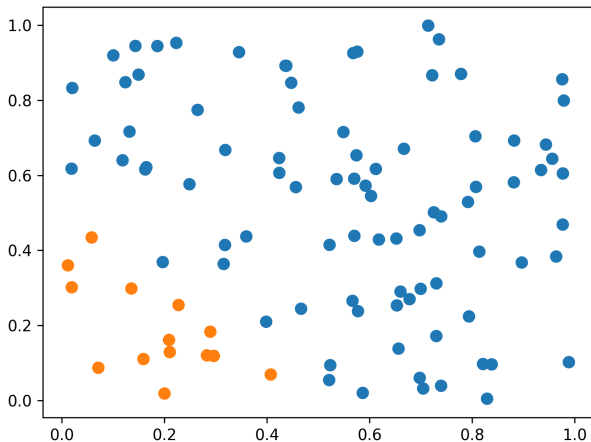


Figure: Points

$x_1, x_2, \dots, x_m \in \mathbb{R}^2$ with
corresponding labels

$y_1, y_2, \dots, y_m \in \{-1, 1\}$ (blue
or orange) with $m = 100$.

Goal: Find a tree classifier to
separate blue and orange points.

Motivating example: Binary classification

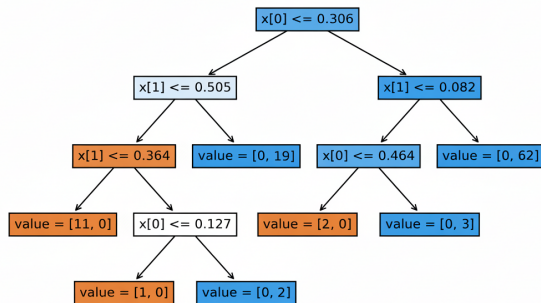


Figure: Tree obtained using python sklearn package.

In our images, $X[0]$ corresponds to the vertical axis and $X[1]$ to the horizontal axis.

An output $value = [a, b]$ means that among the (initially $m = 100$) sample points which made it into this node, there are a orange points and b blue points.

Motivating example: Binary classification

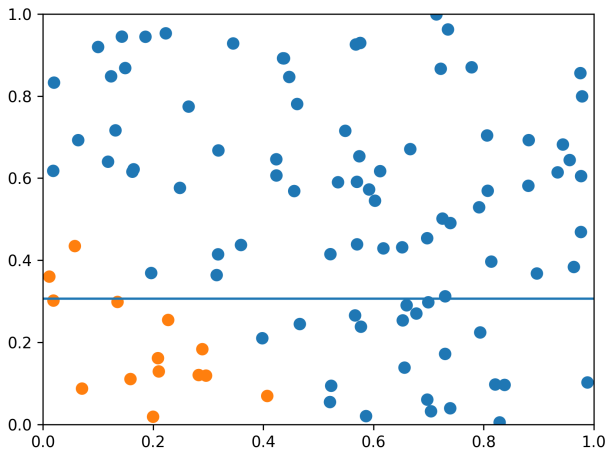


Figure: We can illustrate how the decision tree iteratively splits up the data.

First cut: $X[0] \leq 0.306$

Motivating example: Binary classification

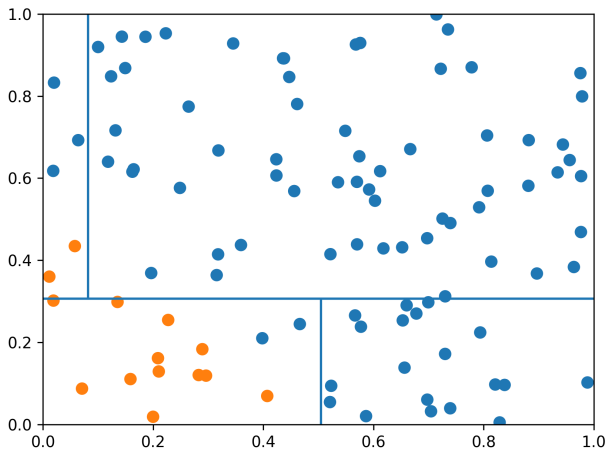


Figure: We can illustrate how the decision tree iteratively splits up the data.

First cut: $X[0] \leq 0.306$

Second round of cuts:

$(X[1] \leq 0.505) \mid (X[0] \leq 0.306)$

$(X[1] \leq 0.082) \mid (X[0] > 0.306)$

Motivating example: Binary classification

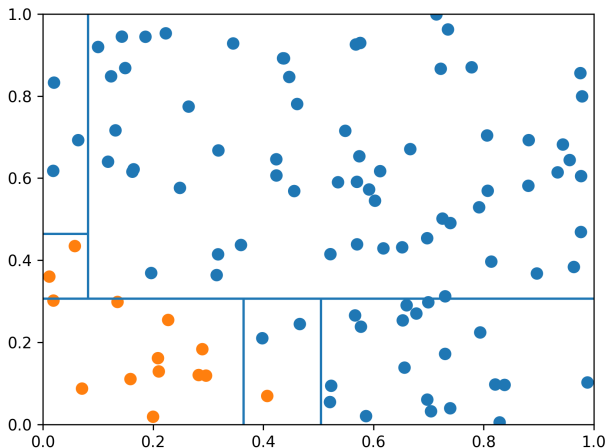


Figure: We can illustrate how the decision tree iteratively splits up the data.

First cut: $X[0] \leq 0.306$

Second round of cuts:

$(X[1] \leq 0.505) \mid (X[0] \leq 0.306)$

$(X[1] \leq 0.082) \mid (X[0] > 0.306)$

Third round of cuts:

$(X[1] \leq 0.364 \mid X[0] \leq 0.306, X[1] \leq 0.505)$

$(X[0] \leq 0.464 \mid X[0] > 0.306, X[1] \leq 0.082)$

Motivating example: Binary classification

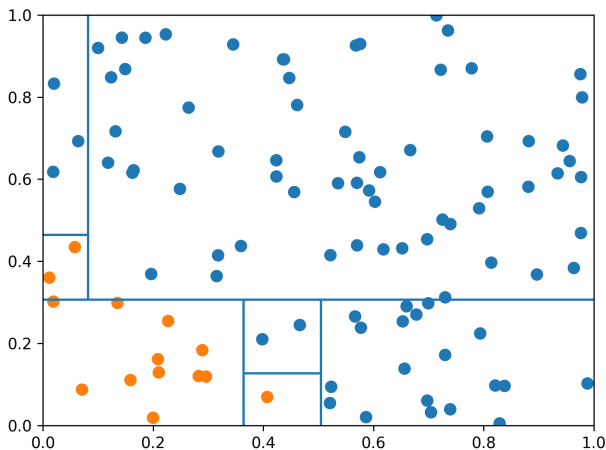


Figure: We can illustrate how the decision tree iteratively splits up the data.

First cut: $X[0] \leq 0.306$

Second round of cuts:

$(X[1] \leq 0.505) \mid (X[0] \leq 0.306)$

$(X[1] \leq 0.082) \mid (X[0] > 0.306)$

Third round of cuts:

$(X[1] \leq 0.364 \mid (X[0] \leq 0.306, X[1] \leq 0.505))$

$(X[1] \leq 0.464 \mid (X[0] > 0.306, X[1] \leq 0.082))$

Final cut:

$(X[1] \leq 0.127 \mid (X[0] \leq 0.306, 0.364 < X[1] \leq 0.505))$

Motivating example: Binary classification

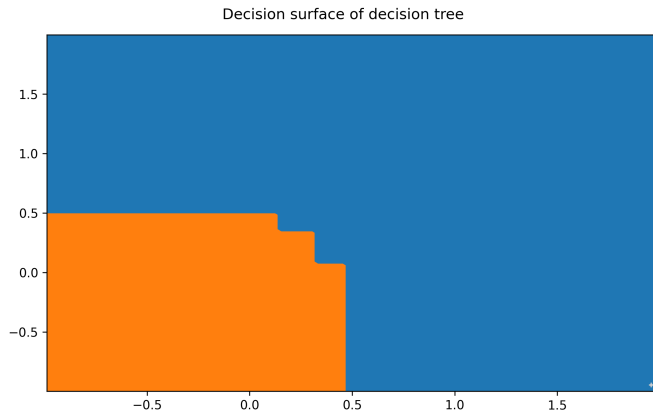


Figure: Overall decision surface of our tree (note that the decision surface is not limited to $[0, 1]^2$).

Motivating example: Binary classification

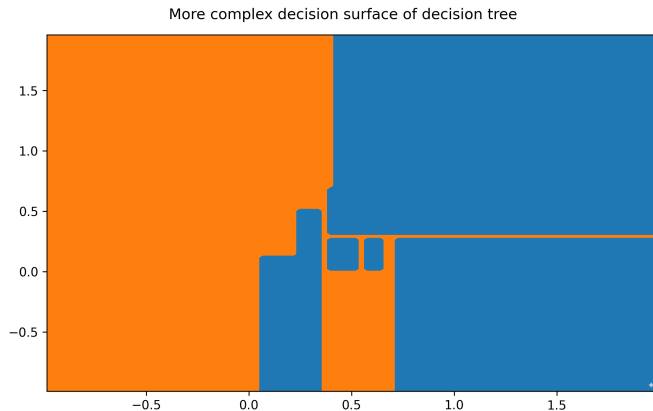


Figure: A different decision surface from a different tree, which emphasizes that decision trees can be very irregular.

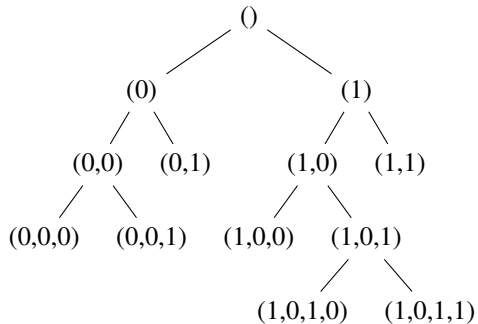
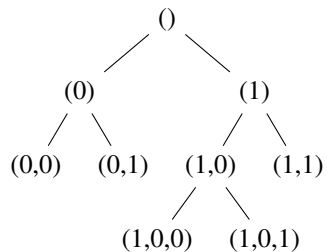
Tree based methods: Advantages and difficulties

- Tree models are *simple* in the way that they transform inputs to outputs
 - Compared to RKHS, the space of functions represented by trees is easy to understand
 - Trees are easier to interpret compared to both kernel methods and neural networks
- The major difficulty in dealing with tree based methods are the optimization methods, i.e., how to fit a tree to data
 - This will be the focus for the majority of these slides! We will study the CART approach to build so-called **standardized binary trees**, which are binary trees where every decision only depends on a single feature
- Our main references are Breiman et al., 1984; Wüthrich and Buser, 2023; Choi, 2017

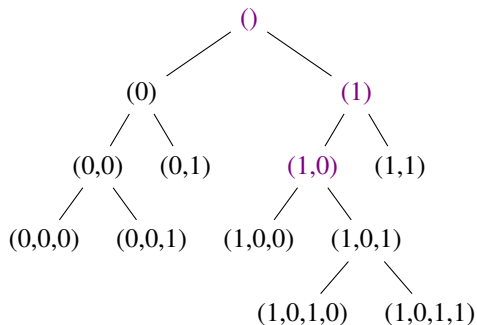
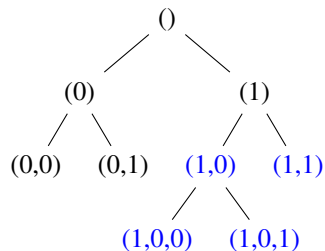
Let $t \in \{0, 1\}^l$, $l \in \mathbb{N}_0 = \{0, 1, 2, \dots\}$. Such t will be the way we label nodes in trees.

- For $l = 0$, $t = ()$ is the empty sequence, which corresponds to the root node of the tree
- We define $t^{flip} \in \{0, 1\}^l$ by $t_i^{flip} = t_i$ for $i < l$ and $t_l^{flip} = 1 - t_l$ (for $l = 0$, $t^{flip} = ()$)
- If $l > 1$, we define $t^{cut} \in \{0, 1\}^{l-1}$ by $t_i^{cut} = t_i$ for $i \leq l - 1$; if $l = 1$, $t^{cut} = ()$
- For $t^1 \in \{0, 1\}^{l_1}$ and $t^2 \in \{0, 1\}^{l_2}$ we say $t^1 \prec t^2$ if $l_1 < l_2$ and $t_i^1 = t_i^2$ for all $i \leq l_1$
→ If $t^1 \prec t^2$ are in the same tree, then t^1 is an ancestor node of t^2 (equivalently: t_2 is a descendant node of t_1).

Notation and illustration of a tree

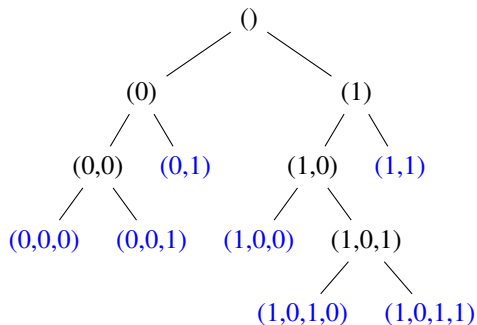
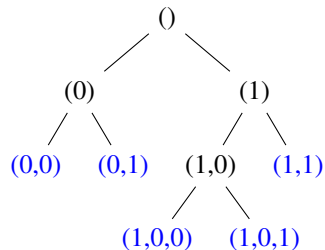


Notation and illustration of a tree



We illustrate the **descendant nodes of (1)** in the left tree and the **ancestor nodes of $(1, 0, 0)$** in the right tree.

Notation and illustration of a tree



Leaves (terminal nodes) of the trees illustrated, which are all nodes which do not have any descendants

Function space corresponding to binary trees

Definition 7.1 (Trees)

A **binary tree** is defined through a triplet (T, P, V) satisfying:

- $T \subseteq \cup_{l \in \mathbb{N}_0} \{0, 1\}^l$ with $1 \leq |T| < \infty$ are the **nodes** of the tree, satisfying $t^{\text{flip}} \in T$ and $t^{\text{cut}} \in T$ for all $t \in T \setminus \{()\}$
- $\mathcal{T} = \{t \in T : t \not\prec \tilde{t} \text{ for all } \tilde{t} \in T\}$ are called the **leaves** of the tree.
- $P = \{P_t \subseteq \mathcal{X} : t \in T\}$ satisfying $P_t \dot{\cup} P_{t^{\text{flip}}} = P_{t^{\text{cut}}}$ for all $t \in T \setminus \{()\}$ and $\dot{\cup}_{t \in \mathcal{T}} P_t = \mathcal{X}$.
- $V = \{\hat{y}_t \in \mathcal{Y} : t \in \mathcal{T}\}$

The function $f : \mathcal{X} \rightarrow \mathcal{Y}$ corresponding to the tree is defined by $f(x) = \sum_{t \in \mathcal{T}} \hat{y}_t \mathbf{1}_{P_t}(x)$ for $x \in \mathcal{X}$.

Note that this definition always implies $() \in T$ and $P_{()} = \mathcal{X}$.

Remark 7.2 (Exercise)

For any finite partition $\mathcal{F}$ of $\mathcal{X}$ ($\dot{\cup}_{F \in \mathcal{F}} F = \mathcal{X}$, $|\mathcal{F}| < \infty$) and values $\{y_F \in \mathcal{Y} : F \in \mathcal{F}\}$, the function $f(x) = \sum_{F \in \mathcal{F}} y_F \mathbf{1}_F(x)$ can be realized as a binary tree function.

From now on, we assume $x = (x^{(1)}, \dots, x^{(d)}) \in \mathcal{X} \subseteq \mathcal{X}_1 \times \dots \times \mathcal{X}_d$.

In standardized trees, each decision is only based on a single feature $j \in \{1, \dots, d\}$, leading to the rectangular structure which we also saw in the motivating examples.

Definition 7.3 (Standard Trees)

A **standardized binary tree** is a binary tree (T, P, V) such that for any $t \in T \setminus \{()\}$, there exists $j \in \{1, \dots, d\}$ and $C_j^t \subseteq \mathcal{X}_j$ such that

$$P_t = P_{t^{cut}} \cap \{x \in \mathcal{X} : x^{(j)} \in C_j^t\}.$$

Furthermore, if there is an ordering $\leq_j$ on $\mathcal{X}_j$, then C_j^t or $(C_j^t)^c$ are of the form $\{x^{(j)} \in \mathcal{X}_j : x^{(j)} \leq c_j^t\}$ for some $c_j^t \in \mathcal{X}_j$.

One usually only works with standardized binary trees to control the complexity of optimizing the trees. However, standardized binary trees are theoretically still a rich class of models:

Remark 7.4 (Exercise continued)

Let $\mathcal{X}_j = \mathbb{R}, j = 1, \dots, d, \mathcal{X}$ be compact and $\mathcal{Y} = \mathbb{R}$. Then functions corresponding to standardized binary trees can approximate any continuous function from $\mathcal{X}$ to $\mathbb{R}$ arbitrarily well w.r.t. $\|\cdot\|_\infty$.

Standard binary tree growing (CART) algorithm

As usual, we have **training data** $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}, i = 1, \dots, m$.

Algorithm Basic structure of tree growing algorithm

- 1: Initialize $T = \{()\}$ and $P_{()} = \mathcal{X}$.
 - 2: **while** Stopping criterion (*) is not reached **do**
 - 3: Choose a node $t \in \mathcal{T}$ ($t \in \{0, 1\}^l$), a variable $j \in \{1, \dots, d\}$ and a set $C_j \subseteq \mathcal{X}_j$ according to criterion (**)
 - 4: Set $T \leftarrow T \cup \{t0, t1\}$, where $t0 = (t_1, t_2, \dots, t_l, 0)$ and $t1 = (t_1, t_2, \dots, t_l, 1)$
 - 5: Set $P_{t0} = P_t \cap \{x \in \mathcal{X} : x^{(j)} \in C_j\}$, $P_{t1} = P_t \cap \{x \in \mathcal{X} : x^{(j)} \notin C_j\}$
 - 6: **end while**
 - 7: Compute the values $V = \{\hat{y}_t : t \in \mathcal{T}\}$; hereby, for all $t \in \mathcal{T}$, $\hat{y}_t$ is calculated solely using the training data $\{y_i : i \in \{1, \dots, m\}, x_i \in P_t\}$ according to criterion (***)
 - 8: Return (T, P, V)
-

What's left is to specify the criteria (*), (**) & (***) !

(***) How to assign values to leaves?

- We start by assigning values to $V = \{\hat{y}_t : t \in \mathcal{T}\}$, that is, with criterion (***) .
- This means, we have our split $\mathcal{X} = \dot{\cup}_{t \in \mathcal{T}} P_t$ and we only need to decide how to assign values $\hat{y}_t \in \mathcal{Y}$ for $t \in \mathcal{T}$.

We start with simple cases which are often used, and then go to the more general rule:

- **Empirical mean:** For regression ($\mathcal{Y} = \mathbb{R}$, usually with the squared loss):

$$\hat{y}_t = \frac{1}{|\{i \in \{1, \dots, m\} : x_i \in P_t\}|} \sum_{i \in \{1, \dots, m\} : x_i \in P_t} y_i$$

- **Majority vote:** For classification ($\mathcal{Y} = \{1, \dots, K\}$):

$$\hat{y}_t \in \arg \max_{c \in \mathcal{Y}} |\{i \in \{1, \dots, m\} : x_i \in P_t, y_i = c\}|$$

- **In general**, we set

$$\hat{y}_t \in \arg \min_{y \in \mathcal{Y}} \sum_{i=1}^m L(y, y_i) \mathbf{1}_{x_i \in P_t},$$

which recovers the empirical mean for $L(y, y') = (y - y')^2$ and the majority vote for $L(y, y') = \mathbf{1}_{y \neq y'}$.

(***) How to assign values to leaves?

We will use the **frequentist viewpoint** and set

$$p(t) = \frac{|\{i \in \{1, \dots, m\} : x_i \in P_t\}|}{m}, \quad p(y | t) = \frac{|\{i \in \{1, \dots, m\} : x_i \in P_t, y_i = y\}|}{|\{i \in \{1, \dots, m\} : x_i \in P_t\}|}.$$

Note: $p(t)$ is the estimated probability of an input x belonging to the set P_t , and $p(y | t)$ the likelihood of y being the output given that an input x belongs to the set P_t .

Probabilistic forecast: In (multi-class) classification, a probabilistic forecast (meaning that the tree function maps onto the space of probability measures on $\mathcal{Y}$) of the tree function can directly be defined through $p(y | t)$: For an input $x \in P_t$, $t \in \mathcal{T}$, the tree assigns probability $p(y | t)$ to $y \in \mathcal{Y}$.

(**) How to grow a tree/split nodes?

- Next, we take a look at how to grow our tree by splitting nodes, i.e., criterion (**).
- **Goal:** Given (T, P) , find a suitable node $t \in \mathcal{T}$, a variable $j \in \{1, \dots, d\}$ and a set $C_j \subseteq \mathcal{X}_j$ such that the tree $(T^{(t,j,C_j)}, P^{(t,j,C_j)})$ resulting from splitting P_t into $P_t \cap \{x \in \mathcal{X} : x^{(j)} \in C_j\}$ and $P_t \cap \{x \in \mathcal{X} : x^{(j)} \notin C_j\}$ is in some sense "optimal"

To this end, for a tree structure (T, P) ,

- we call $I(t) \in [0, \infty)$ the **impurity of node** $t \in T$, and
- $\bar{I}(T) = \sum_{t \in \mathcal{T}} p(t) I(t)$ the **impurity of T**.

Idea: Lower impurity is better. Thus, the optimal split is one such that

$$(t, j, C_j) \in \arg \min_{(t', j', C_{j'})} \bar{I}(T^{(t', j', C_{j'})})$$

In practice: One usually does not need to optimize the choice of $t \in \mathcal{T}$, since the stopping criterion can be applied locally (see later), and thus one simply keeps splitting each $t \in \mathcal{T}$ until the stopping rule applies.

(**) How to grow a tree/split nodes? Classification

How to choose the **impurities** $I(t) \in [0, \infty)$ of nodes t and $\bar{I}(T) = \sum_{t \in T} p(t)I(t)$ of T ?

For **classification** ($\mathcal{Y} = \{1, \dots, K\}$), one usually sets

$$I(t) = \phi\left(p(1 | t), p(2 | t), \dots, p(K | t)\right),$$

where $\phi : \{p \in [0, 1]^K : \sum_{k=1}^K p_k = 1\} \rightarrow \mathbb{R}_+$ is called an **impurity function**.

Example 7.5

- **Gini impurity:** $\phi(p_1, \dots, p_K) = \frac{1}{2} \sum_{k=1}^K p_k(1 - p_k)$
- **Entropic impurity:** $\phi(p_1, \dots, p_K) = - \sum_{k=1}^K p_k \log(p_k)$ (as usual, $0 \log(0) = 0$)
- **Misclassification rate impurity:** $\phi(p_1, \dots, p_K) = 1 - \max\{p_1, \dots, p_K\}$ or more generally $\phi(p_1, \dots, p_K) = \min_{k=1, \dots, K} \sum_{l=1}^K L(k, l) p_l$ (We note that choosing $L(k, l) = \mathbf{1}_{k \neq l}$ recovers the case $\phi(p_1, \dots, p_K) = 1 - \max\{p_1, \dots, p_K\}$)

(**) How to grow a tree/split nodes? Regression

How to choose the **impurities** $I(t) \in [0, \infty)$ of nodes t and $\bar{I}(T) = \sum_{t \in T} p(t)I(t)$ of T ?

For **regression** ($\mathcal{Y} = \mathbb{R}$), one usually sets, with $m_t := |\{i \in \{1, \dots, m\} : x_i \in P_t\}|$,

$$I(t) = \text{Var}(\{y_i : i \in \{1, \dots, m\}, x_i \in P_t\}) = \frac{1}{m_t} \sum_{i: x_i \in P_t} \left(y_i - \frac{1}{m_t} \sum_{l: x_l \in P_t} y_l \right)^2$$

(**) How to grow a tree/split nodes? Impurity and loss functions

The impurity determines the loss function which the overall tree function wants to minimize.

Lemma 7.6

- In **regression** with empirical mean rule, $I(t) = \text{Var}(\{y_i : i \in \{1, \dots, m\}, x_i \in P_t\})$ corresponds to the square loss:

$$\bar{I}(T) = \frac{1}{m} \sum_{i=1}^m (f(x_i) - y_i)^2$$

- In **classification** (with probabilistic forecast $f(x, c) = p(c \mid t)$ for $x \in P_t$), the entropic impurity corresponds to log-loss (or cross-entropy loss):

$$\bar{I}(T) = -\frac{1}{m} \sum_{i=1}^m \log(f(x_i, y_i))$$

Note: Impurities are used as a more local concept of a loss, so that we can quantify the contribution of each node towards the overall loss.

(*) Stopping criterion/regularization

- So far, we have seen that we want to keep splitting a tree to reduce impurity (which basically means: we keep splitting to reduce our loss)
- If we just kept splitting all the way until no further improvement is possible, this would mean that we arrived at zero loss (overfitting!)
- The way to avoid overfitting in CART is quite ingenious due to its simplicity: We simply choose a hyperparameter $\alpha > 0$ and set

$$I_{\alpha}(t) := I(t) + \frac{\alpha}{p(t)}$$
$$\bar{I}_{\alpha}(\mathbf{T}) := \sum_{t \in \mathcal{T}} p(t) I_{\alpha}(t) = \bar{I}(\mathbf{T}) + \alpha |\mathcal{T}|$$

→ we **penalize the number of leaves in the tree**

→ the **penalization is locally defined** for each node t

- We split a certain node $t \in \mathcal{T}$ if and only if it leads to a decrease in $\bar{I}_{\alpha}$, i.e., if

$$\left(\min_{(j', C_{j'})} \bar{I}_{\alpha} \left(\mathbf{T}^{(t, j', C_{j'})} \right) < \bar{I}_{\alpha}(\mathbf{T}) \right) \Leftrightarrow \left(\min_{(j', C_{j'})} \bar{I} \left(\mathbf{T}^{(t, j', C_{j'})} \right) < \bar{I}(\mathbf{T}) - \alpha \right)$$

(*) Stopping criterion/regularization

Recall the notation:

- Given (T, P) , $t \in \mathcal{T}$, $j \in \{1, \dots, d\}$ and $C_j \subseteq \mathcal{X}_j$, the tree $(T^{(t,j,C_j)}, P^{(t,j,C_j)})$ is defined as the one resulting from splitting P_t into $P_t \cap \{x \in \mathcal{X} : x^{(j)} \in C_j\}$ and $P_t \cap \{x \in \mathcal{X} : x^{(j)} \notin C_j\}$
- If $t \in \{0, 1\}^l$, set $t0 = (t_1, t_2, \dots, t_l, 0)$ and $t1 = (t_1, t_2, \dots, t_l, 1)$
- $P_{t0} = P_t \cap \{x \in \mathcal{X} : x^{(j)} \in C_j\}$, $P_{t1} = P_t \cap \{x \in \mathcal{X} : x^{(j)} \notin C_j\}$

The splitting criterion for node $t \in \mathcal{T}$, namely

$$\left(\min_{(j', C_{j'})} \bar{I}_\alpha \left(T^{(t,j',C_{j'})} \right) < \bar{I}_\alpha(T) \right) \Leftrightarrow \left(\min_{(j', C_{j'})} \bar{I} \left(T^{(t,j',C_{j'})} \right) < \bar{I}(T) - \alpha \right)$$

can be expressed locally as

$$\min_{j', C_{j'}} p(t0) I(t0) + p(t1) I(t1) < p(t) I(t) - \alpha.$$

(Note that $I(t0)$ and $I(t1)$ implicitly depend on j' and $C_{j'}$ through the definition of P_{t0} and P_{t1})

(*) Stopping criterion/regularization

Question: How to choose the penalty parameter α ?

- The simple answer is: cross-validation
- Straightforward cross-validation is not the most efficient way though
- We can exploit the following: The choice of α only influences *how many* splits there are (size of the tree), but not *what kind* of splits (structure of the tree)
- Efficient way for cross validation: Fit the largest tree (corresponding to $\underline{\alpha} = 0$). Then simply **prune** this tree (iteratively remove pairs of leaves $\{t0, t1\} \in \mathcal{T}$) until it is optimal for a given level of $\alpha > 0$, meaning that we remove pairs of leaves $\{t0, t1\} \in \mathcal{T}$ for which

$$\min_{j', C_{j'}} p(t0) I(t0) + p(t1) I(t1) \geq p(t) I(t) - \alpha.$$

- For cross validation over possible penalty parameter values $\bar{\alpha} = \alpha_1 \geq \dots \geq \alpha_N = \underline{\alpha} = 0$ we simply have to grow the tree one time (up to level $\underline{\alpha}$) and prune it one time (up to level $\bar{\alpha}$)
- Pruning iteratively removes those binary splits whose deletion increases the impurity of the tree the least

Solving the optimization problem for optimal splits

How to solve the optimization problem to obtain an optimal feature j and cut C_j for a given node t ?

$$\min_{j', C_{j'}} p(t0) I(t0) + p(t1) I(t1)$$

The difficulty of this optimization problems strongly depends on the types of features used.

Interlude: Different kinds of features

There is

- Numerical data; either continuous (e.g., blood sugar level, CO2 percentage in the air, precise age) or discrete (e.g., number of children, rolling of a die)
- Categorical data (e.g., color of car, blood type, gender, country of origin)

Categorical data is often hard to deal with in practice. To make it more feasible, one can:

- Use **one-hot encoding**. For categories $V = \{v_1, \dots, v_K\}$, the encoding $\phi : V \rightarrow \mathbb{R}^K$ maps v_k to the k -th standard basis vector in $\mathbb{R}^K$, $k = 1, \dots, K$.

Purpose: Can use categorical data as numerical input to commonly used models. And the encoded variables are very easy to separate in $\mathbb{R}^K$, even linearly.

- Use **target encoding**. Idea is to add to each category a certain influence that it has on a target (often: the prediction target). E.g., if categories are brands of cars, a possible target encoding is to add to each brand the average car price among this brand.

Purpose: This is a natural way to encode the importance of categorical features on some target, and to obtain an artificial ordering among the different categories (i.e., Audi $\geq$ VW or similar).

Solving the optimization problem for optimal splits

How to solve the optimization problem to obtain an optimal feature j and cut C_j for a given node t ?

- **Numerical features:** If we have an ordering $\leq_j$ on $\mathcal{X}_j$ and a vector space structure on $\mathcal{X}_j$, then it is simple: We simply order the values in $\{x_i^{(j)} : i = 1, \dots, m, x_i \in P_t\}$, say $x_{(1)}^{(j)} \leq_j x_{(2)}^{(j)} \leq_j \dots \leq_j x_{(n_{t,j})}^{(j)}$ with $n_{t,j} := |\{x_i^{(j)} : i = 1, \dots, m, x_i \in P_t\}|$, and only consider possible cuts

$$C_j^{(n)} = \left\{ \tilde{x}^j \in \mathcal{X}_j : \tilde{x}^j \leq \frac{x_{(n)}^{(j)} + x_{(n+1)}^{(j)}}{2} \right\}, \quad n \in \{1, \dots, n_{t,j}\}.$$

A brute force search over $C_j^{(n)}$ only takes $n_{t,j}$ many steps $\rightarrow$ **Feasible**.

- **Categorical features:** If we have no ordering, then we have $(2^{n_{t,j}-1} - 1)$ ways to choose a set $C_j \rightarrow$ **Infeasible**.
- For **categorical features**, one thus requires a heuristic approximation. A frequently used approach is (local) target encoding. That is, we define an artificial numerical ordering on the data. E.g., one can define an ordering on $\{x_i^{(j)} : i = 1, \dots, m, x_i \in P_t\}$ via the number of times each category occurs in the training data.

Implementation

- Standard python package: `sklearn.tree`
- As mentioned before, the usual ways to make trees more powerful are based on model aggregation/ensemble learning methods, which we will see in the upcoming lectures

Takeaway messages:

- We saw that trees are easy to interpret and simple classes of functions
 - ① In particular, one usually only works with binary trees
 - ② Also, most trees are standardized, i.e., each split only depends on a single feature
 - ③ Trees are still powerful and can approximate most functions
- The major difficulty is to fit a tree to data, where we looked at the tractable and frequently used CART approach:
 - ① How to assign values to terminal leaves
 - ② How to choose a standardized binary split for a given tree/node (impurity functions)
 - ③ How to choose a stopping criterion (regularization and pruning)

List of references I

Breiman, L et al. (1984). *Classification and Regression Trees*. Wadsworth.

Choi, Hyeong In (2017). *Classification and Regression Tree (CART)*. Lecture notes, Seoul National University. URL: <https://www.math.snu.ac.kr/~hichoi/machinelearning/lecturenotes/CART.pdf>.

Wüthrich, Mario and Christoph Buser (2023). *Data Analytics for Non-Life Insurance Pricing*. Lecture notes, ETH Zürich. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2870308.